Stock & Foil

Private collateral registry for receivables finance · built on Midnight

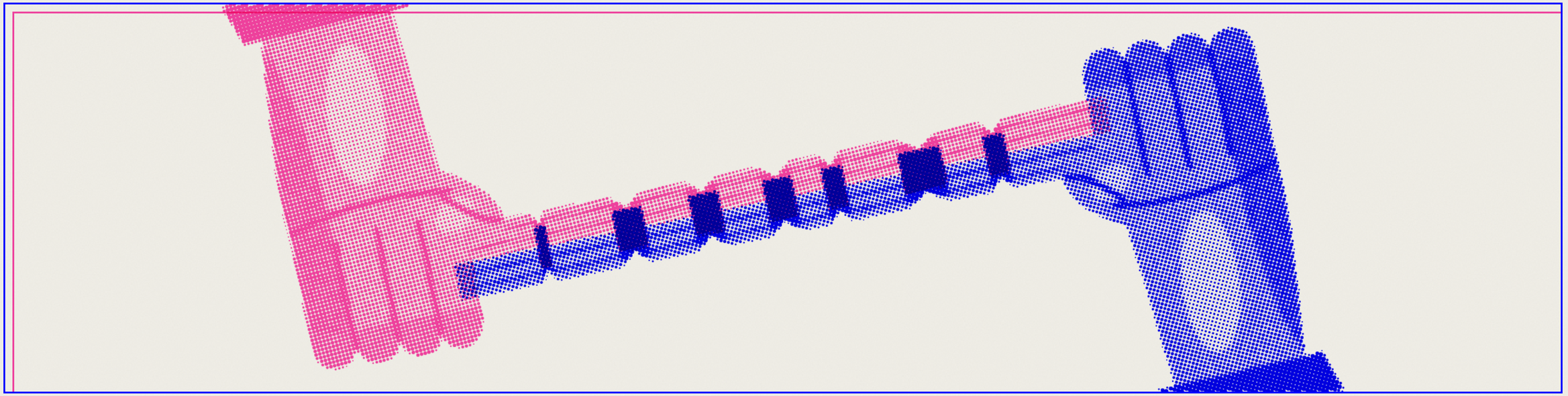
Every invoice can be pledged once.

Prove it without revealing it.

A credit officer at a factoring company funds an invoice on the borrower's word that nobody else is holding it. Stock & Foil answers that question — yes or no — before the money leaves, and neither lender shows the other anything.

new in Wave 2

- **Two registries,** two operators, two ceremonies, both live on preview — and a lender's check across them `slide 07`
- **A wallet holding zero DUST transacted,** so a buyer's clerk needs no token `slide 08`
- **Four new circuits, 12 → 16;** tests 177 → 388; CI on every push `slides 12–14`
- **Who benefits, named** — and every Wave 1 note answered `slides 03, 17`



Stock, kept by the lender · Foil, kept by the buyer · notches cut from 0x31e5…46f7

Compact 0.31.1 · 16 circuits · 388 tests · stock-and-foil.vercel.app, no wallet needed

Two lenders, one invoice, nobody able to check.

September 2025 · First Brands Group · Chapter 11
Source: Global Trade Review, on the receivables and inventory finance fraud alleged against the founder.

First Brands filed for Chapter 11 carrying roughly **$2.3 billion** of receivables-finance liabilities. The invoices behind them are alleged to have been forged, inflated tenfold, and presented to more than one financier.

Three frauds, one shape: a receivable that is not what the lender was told it was, or is not the lender's to hold. On paper, each is indistinguishable from an honest one — and **none of the financiers could have caught it by comparing notes,** because there was no way to compare notes that cost less than the fraud.

$2.3B

of receivables-finance liabilities at filing

10×

the multiple some invoice values were inflated by

>1

financiers shown the same invoice

The registry model is already proven, and the disclosure problem is what caps it. Singapore's Trade Finance Registry has a national mandate and 40+ participating institutions, and in December 2025 *Global Trade Review* still reported that its effectiveness "has been limited because the relatively low number of transactions being submitted into the system means it may not detect duplicate financing". Submitting means disclosing your book.

GTR, 16 Dec 2025
Singapore's anti-fraud trade registry eyes expansion

Stock & Foil lets them check without showing each other anything.

Who stands to benefit.

Four jobs, one receivable. For each: what they are stuck with today, what the registry hands them, and the thing they stop having to send to anybody.

> *"Its content accurately addressed the pain points. The presentation would be even better if it made it clearer who stands to benefit from using this."*
>
> Wave 1 review · Communication. This slide is the answer.

01

The credit officer

at a factoring company · Meridian Factoring

Funds the invoice on the borrower's word that nobody else is holding it.

what they get

A yes or no on whether this receivable is already pledged — before the money leaves.

what they stop sending

Circulating the invoice, the debtor or the client list to a rival lender to find out.

what does it

`ALREADY_ENCUMBERED` and the cross-registry check, slide 07

02

The CFO

of the supplier · Kestrel Components

Pays a fraud premium set by what the lender cannot check.

what they get

An invoice a financier can confirm is unencumbered, priced on the receivable and not the doubt.

what they stop sending

Handing the whole debtor-by-debtor ledger to every lender that asks to see the book.

what does it

`certifyBorrowingBase` a pool proved over a floor, slide 11

03

The accounts-payable team

at the buyer · Ardmore Retail Group

Verifies the same invoice by telephone for every financier in the chain.

what they get

One acknowledgment, made once, and payment that reaches whoever holds the debt now.

what they stop sending

Confirming payables to callers it has no way to authenticate.

what does it

`acknowledge` with the fee sponsored, so no token, slide 08

04

The insolvency practitioner

appointed over the estate · Halloran & Reeve

Rebuilds who was owed what from six banks' paperwork, months after the money went.

what they get

One record opened under a case reference, with two of three keyholders' approval on the ledger.

what they stop sending

Chasing every financier in the chain to reconstruct a pledge history by hand.

what does it

`openCase` four records, two approvals, slide 10

Each of the four has a workspace in the app — the same registry, seen from that seat, with what the party can and cannot see printed beside it.

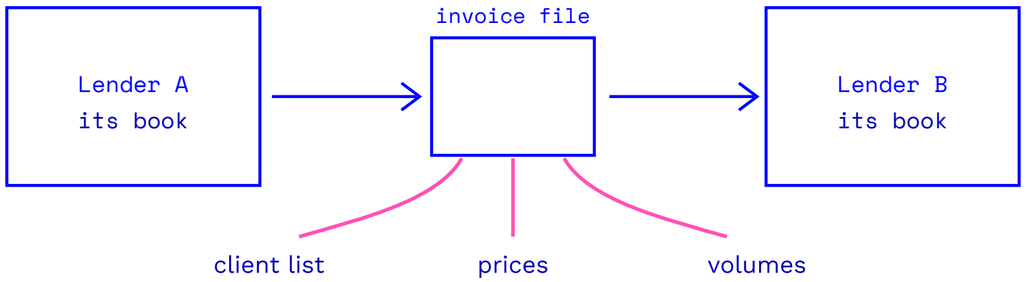
stock-and-foil.vercel.app/#who
/app/financier-b · /app/seller · /app/debtor · /app/auditor

Both fixes cost more than the fraud.

There are exactly two ways to check whether a receivable is already financed, and receivables finance has rejected both for the same reason.

Option one

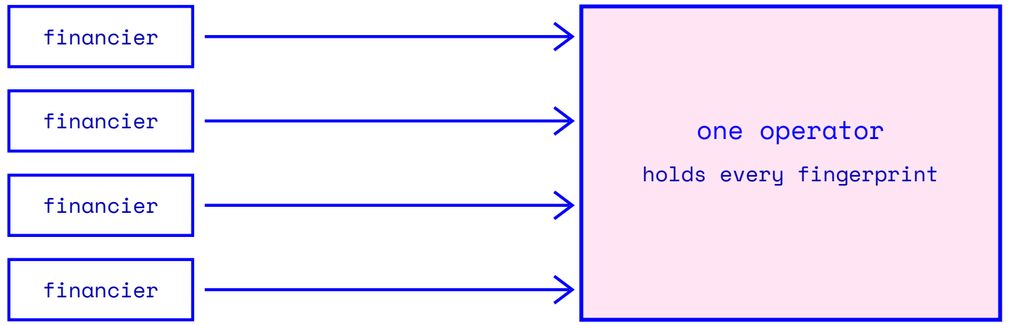
Show the other lender your invoices



An invoice file is a client list and a price list. Handing it to a rival costs more than the fraud does, so every financier keeps its book to itself — and the same receivable gets funded twice.

Option two

Put every fingerprint in one registry

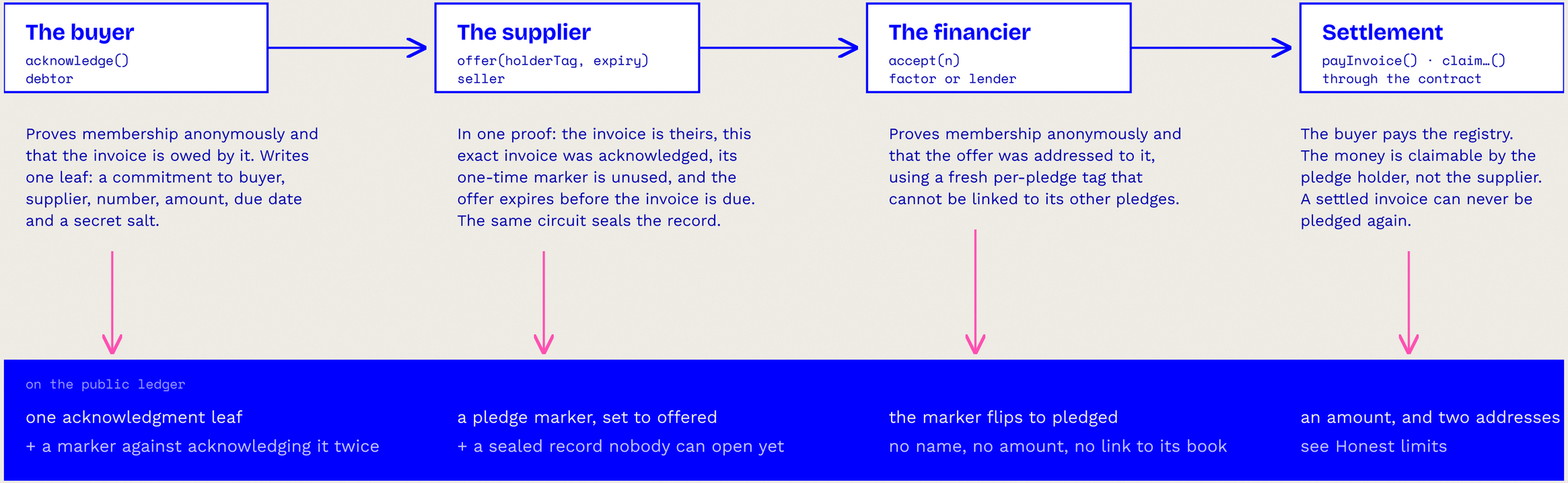


Central fingerprint registries do work, and they ask every financier to trust one operator with the shape of its whole book — who its clients are, how much it lends, and when.

Midnight's dual ledger removes the trade-off. The checks run in zero knowledge against private state, and the public ledger holds only what uniqueness needs: a commitment, a one-time marker, a sealed record.

Four parties, one record.

A receivable passes through four hands. Each hand proves something, and leaves a mark nobody else can read.



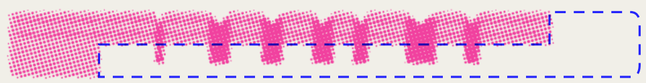
Every check runs while the transaction is being built. A fraudulent pledge cannot be proved, so it never reaches the chain at all — there is no bad state to roll back and no failed transaction to read.

Three refusals.

A tally stick was split so that one half could not be forged against the other. These are the same idea, proved rather than whittled. The assert message in the contract is the product's refusal code, and the SDK and the interface carry it through unchanged.

A forged invoice

Financing raised against an invoice the buyer never owed.

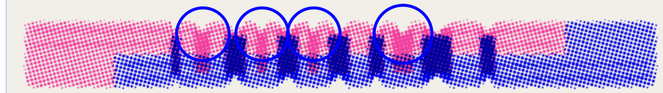


`NOT_ACKNOWLEDGED`

No acknowledgment leaf exists for this invoice, so the offer has no membership proof to make. There is no second half for it to match.

An inflated invoice

The buyer owes $2,300. The financier is shown $23,000.

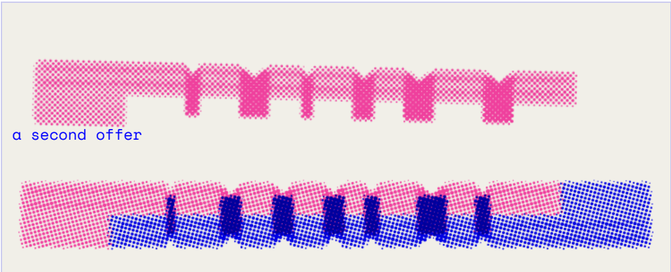


`NOT_ACKNOWLEDGED`

The amount is inside what the buyer signed. A $23,000 invoice is a different commitment from the $2,300 on record, and nothing acknowledges it.

A double pledge

One receivable financed by several financiers at once.



`ALREADY_ENCUMBERED`

The one-time marker for this receivable is already spent. A stick that has been split cannot be split again.

The refused lender learns exactly one bit: this receivable is already encumbered. Not by whom, not for how much, not since when — and the existing pledge is neither touched nor disclosed.

Two registries. One answer.

A pledge marker is a hash of the invoice and its parties alone — no registry identity, no operator key, no deployment salt. So the same receivable has the same marker in *every* registry, and asking another one is a read of public state, not a business relationship.

> *"The catch is scale: it only works once many competing financiers join the same registry, a consortium sale that's hard for a startup."*
>
> Wave 1 review · Product. Below is the answer, run rather than argued.



Read live from both deployed contracts in the visitor's own tab at **stock-and-foil.vercel.app/#check** — no proxy, no API key, no wallet. A registry that does not answer prints *no answer*, never *free*.

Two deployments, nothing shared

A · Stock & Foil	74fe0bf9…8a0fad
B · Harbour	2d8f8f23…0cccbd
Operators	79f27d5f… · 54f28d2d…
Ceremonies	2, independent
Peer declarations	mutual, on chain
Marker for #2002	identical in both

Eleven transactions each, three keyholders each, and neither can read the other's private state.

What the lender asked, and when

#2002 · asked before Harbour had ever seen the receivable

FREE never seen it — Harbour · its own book

ENCUMBERED pledged here — Stock & Foil · a peer

Only the peer could answer, and the lender declined on that one bit. Not the holder, not the amount, not since when.

control · #772002, pledged nowhere

FREE in both. The check says no when the answer is no.

the part that does not flatter us

Harbour accepted a receivable already pledged in Stock & Foil.

Harbour's ledger had never heard of the pledge, so its contract took the offer at block 1322275 — and no contract could have done otherwise, because one ledger cannot read another. Offered a second time *inside* Harbour it is refused on the spot, `ALREADY_ENCUMBERED`.

That gap is the entire reason the check above exists — and it is written into PROOF.md rather than left out of it.

Why the marker travels

These are the keys the two ledgers actually store the entry under, read back from each contract — not one hash computed twice in one process. And it still cannot be guessed: the fingerprint carries the seller's secret salt.

and before any of that

One lender alone is already worth it

A forged and an inflated invoice are refused by the *buyer's* acknowledgment, not by other lenders. One factoring house running its own registry catches both on day one, plus duplicates across its own branches, with nobody else on the network.

registerPeer is a declaration, not a proof that a peer is honest.

Nobody has to buy a token first.

Every Midnight transaction pays its fee in DUST; DUST is generated only by registered NIGHT, is non-transferable, and a contract cannot hold any. Taken literally, that means a buyer's accounts-payable clerk acquires a crypto token before confirming that an invoice is real.

The split, and what it costs each side

the official sponsorship pattern

The acting party
0 NIGHT · 0 DUST

proves the circuit
balances the shielded and unshielded sides
signs, and binds

→

The sponsor
a funded wallet beside it

balances the DUST
submits the transaction
pays 1 SPECK

→

The chain
preview, block 1321650

accepted, and the acting wallet's balance is still exactly zero

what the sponsor never sees
the invoice · the parties · the amount · which record was sealed

The registry authenticates by knowledge of a secret whose hash is on the ledger — never by who paid the fee. So the sponsor's money buys a block slot and nothing else, and sponsoring somebody weakens no claim the contract makes about them.

The service, and the two reviews it had to pass

sponsor/ · 87 tests

A synchronous budget reservation that holds under twenty parallel requests, so a burst cannot overspend it. A rate-limit bucket keyed by API key or socket address rather than a spoofable `X-Forwarded-For`. And a suite that asserts what the service is allowed to say: no upstream error text and no balance ever reaches a caller.

It has actually happened

A wallet with no DUST transacted on preview.

Acting wallet	mn_addr_preview1h757pw…y42wwl
Its DUST before	**0 SPECK**
Its DUST after	**0 SPECK**
Its NIGHT	0 STAR · 0 registered for DUST
Circuit it ran	acknowledge
Transaction	0xa46d2c3a01…d06e07
Block	1321650
Fee, paid by the sponsor	1 SPECK
Run at	16 Oct 2026, 11:34

Read from the chain *before* the run — the command aborts unless the balance is exactly zero — and read again afterwards. The claim is not that the wallet ought to have been empty but that it was, at the moment it transacted.

What the ledger shows, and what it never shows.

Printed on the ledger public

- **Acknowledgment leaves.** One per acknowledged invoice. The leaf does not say which.
- **Pledge markers and their status.** Offered, pledged, released or settled.
- **Per-pledge holder tags.** A fresh tag for every pledge.
- **Sealed records and their ephemeral keys.**
- **Certificate floors, pool size and validity.**
- **Disclosure requests, cases and approvals.**
- **Peer declarations.** Which other registries this operator recognises.
- **Settlement amounts and addresses,** unshielded.

Settlement is the one exception, and it is published under **Honest limits** rather than hidden.

Never printed stays with the parties

- **The invoice.** Buyer, supplier, number, amount, due date.
- **Which invoice a marker belongs to.**
- **Which financier holds which pledge,** or how big its book is.
- **The record contents,** without 2 of 3 keyholders.
- **The receivables inside a borrowing base.**
- **Anything the auditor learns,** to anyone else.
- **Who was refused,** and what they were trying to pledge.

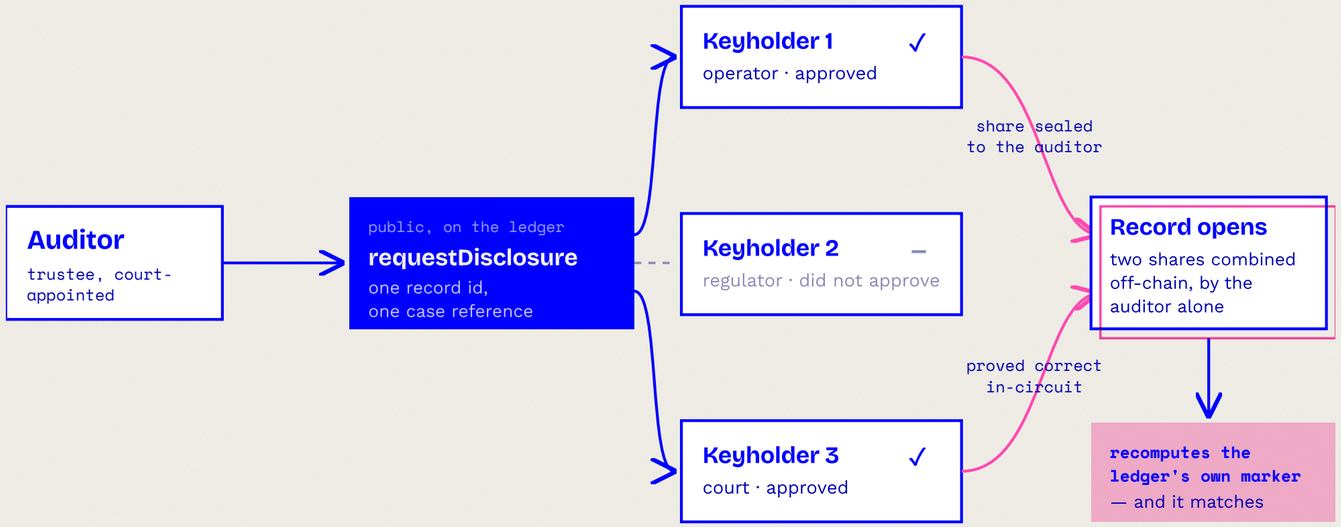
The supplier salts the invoice before the buyer acknowledges it, so a financier who guesses an invoice still cannot test whether it is encumbered.

The explorer in the app reads the same projection an indexer does — and this band is the whole of it. Counts, markers, statuses, ciphertexts. There is nothing underneath to be careless with.

1	2	3	3	3	1	1	2
Debtors	Financiers	Acknowledgments	Pledge markers	Sealed records	Certificates	Disclosure requests	Sealed shares

Disclosure nobody can do alone.

An insolvency practitioner has to be able to open a specific pledge. One party being able to do that would undo everything else.



the key

A 2-of-3 Shamir sharing of the disclosure key over the Jubjub scalar field. The constructor checks the sharing itself, in point form, and refuses a ceremony that would let one party read everything or no two read anything.

the share

A keyholder's share of the record key is computed inside the circuit and sealed to the auditor's key. No keyholder ever publishes anything another keyholder can use.

the check

The opened record recomputes the invoice fingerprint, the acknowledgment leaf and the pledge marker, so a doctored disclosure fails against the ledger's own value. Request and approvals stay public: no record is read unseen.

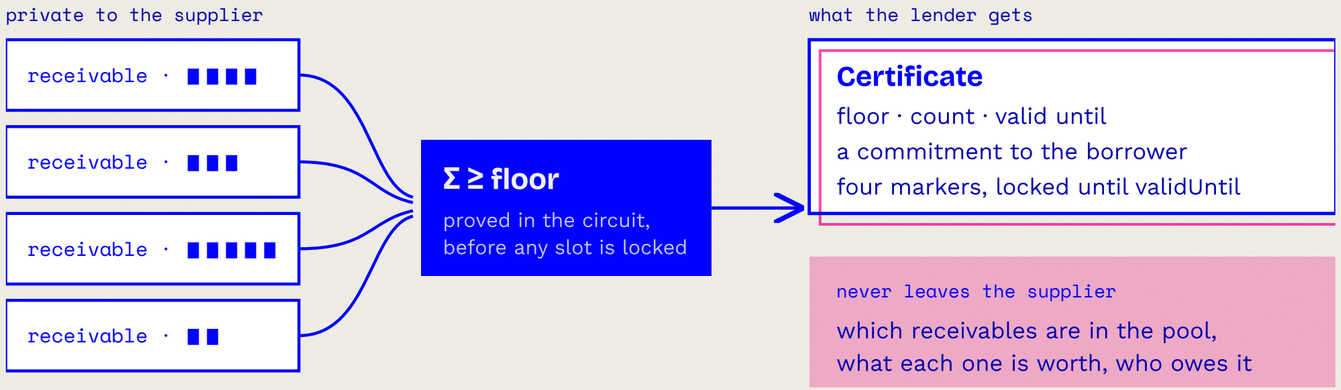
the case **NEW IN WAVE 2**

An insolvency over four receivables used to need eight approvals. **openCase** names them under one reference and **approveCase** seals one share per record in a single call — two approvals, not eight. A share is bound to its case, so an approval cannot be replayed against a record the keyholders never saw.

Preview run · approvals [true, false, true] · the opened record recomputed marker 4fee3054…0aae2a, the key the ledger stores the pledge under.

Prove the pool clears the floor.

An asset-based lender advances against a borrowing base. Today it gets a spreadsheet of line items and takes the borrower's word for the rest.



Each locked slot becomes an ordinary offer expiring with the certificate, which the lender takes up one at a time with the same **accept** it uses for a single invoice — or all at once with **acceptBatch**, new this wave, which proves membership once for the whole batch and either takes every offer or none. Distinctness and the floor are proved before any slot is locked, so a pool that lists one invoice twice is refused for the duplicate rather than for an encumbrance it caused itself.

the certificate's own refusals, asserted in this order

`BAD_EXPIRY` `DUPLICATE_CERTIFICATE` `EMPTY_POOL` `NOT_INVOICE_OWNER` `NOT_ACKNOWLEDGED` `INVOICE_OVERDUE`
`DUPLICATE_INVOICE` `BELOW_FLOOR` `ALREADY_SETTLED` `ALREADY_ENCUMBERED` `BAD_EPHEMERAL`

Four receivables, and on a real network exactly four

Each proved acknowledged, unencumbered, not yet settled and not overdue — the same four checks a single pledge passes. A partly filled pool passes the simulator and *fails on a live network*: the proof server rejects the zeroed curve point an unused slot leaves in the public transcript. Found by running it, recorded in **`PROOF.md`**.

The lock, and what it does not prove

A certificate that only asserted a total would be a promise. This one writes four pledge markers in the same transaction. What no circuit can check is that those markers are addressed to the lender the certificate names: a holder tag is derived from the lender's own secret, so the contract has nothing to test the seller's claim against. The lender recomputes every tag from public state before money moves.

Invoice	Amount	Due	Acknowledged	Encumbrance	Pledge marker
SF-2026-1001 The receivable the whole replay turns on.	$2,300.00	14 Dec 2026	yes	settled	0x951fd1…d11f
SF-2026-1002 Held back for the borrowing base.	$1,200.00	29 Nov 2026	yes	encumbered	0x5c3280…6c38
SF-2026-1003 Held back for the borrowing base.	$900.00	14 Nov 2026	yes	encumbered	0x7eff23…08d6

The supplier's own book, in the app. The registry never holds an invoice — only a hash of the whole document.

Sixteen circuits, one contract.

Compact 0.31.1, language 0.23. Authorisation is always knowledge of a secret whose hash is on the ledger, never the caller's key. Four circuits are new this wave, in pink.

The dual ledger

public state, readable by anyone

- Two membership trees, depth 10
- The acknowledgment tree, depth 16, and its nullifier set
- Pledge markers, keyed by nullifier
- Sealed records, certificates, disclosure requests, cases and shares
- The peers this operator has declared
- The ceremony: disclosure key, three keyholder keys, auditor key

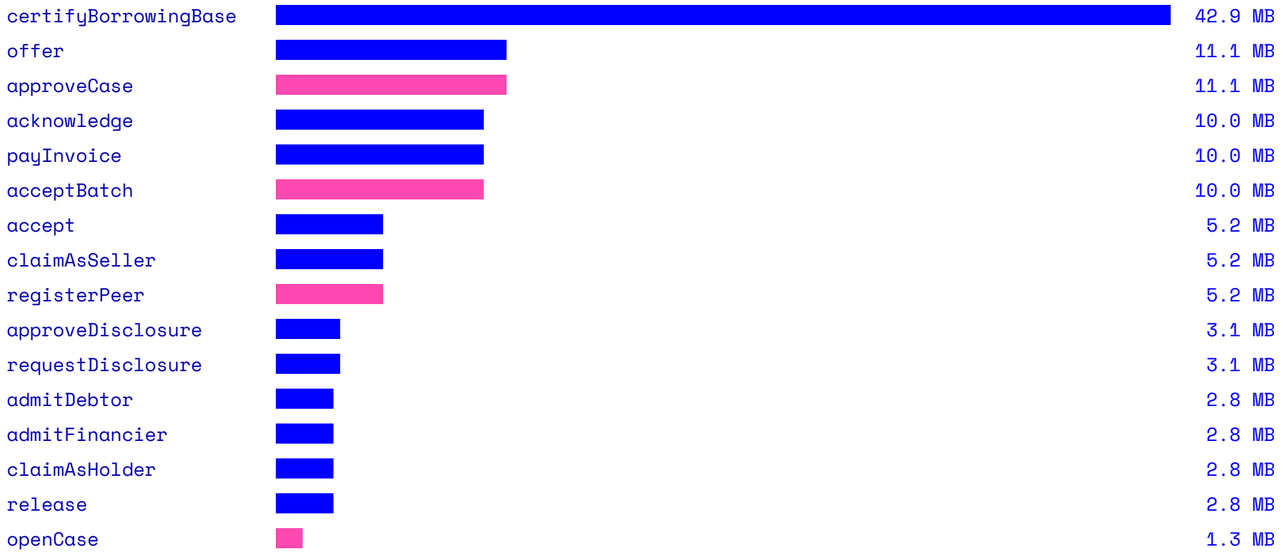
private state, never leaves the party

- The invoice and its salt
- Each party's secret, and the Merkle paths it proves with
- Holder tags, sealing scalars, keyholder shares

Exported circuits	16 · 4 new
Refusal codes, asserted in order	39
Domain separators, one per purpose	17
Compile with proving keys	3 m 20 s

Prover keys

SHA-256 dominates key size and proof time



The seal

A fresh scalar e per record. `E = e·G` is published; the mask is `Poseidon(S.x, S.y, j)` over `S = e·disclosurePk`. Poseidon because a SHA-derived mask sits below 2^{248} and would leak the high bits of the identities it pads.

The deploy

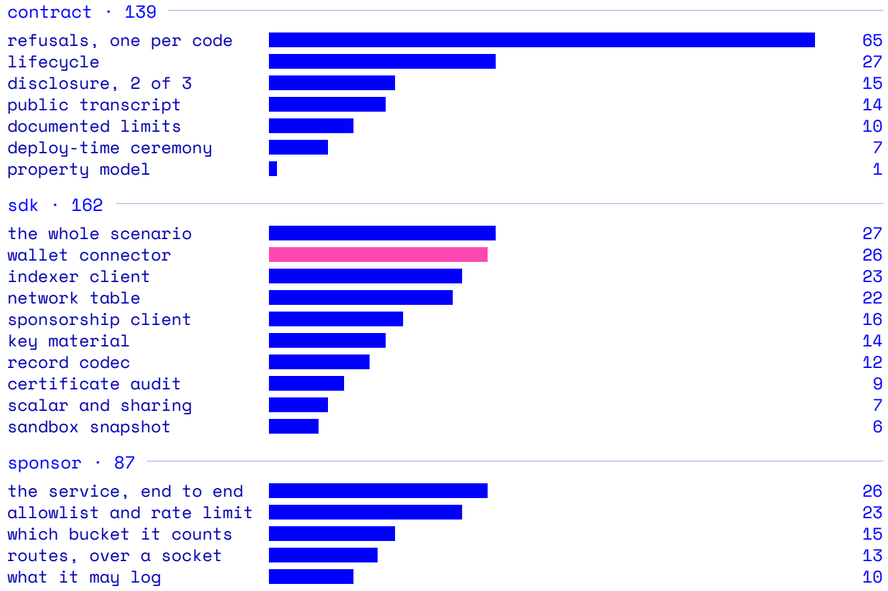
Sixteen verifier keys total about 34 kB; one transaction's byte budget takes six. The deploy carries six, then ten `VerifierKeyInsert` transactions carry the rest — eleven per registry, twenty-two for the two.

388 tests, and the ones that are meant to fail.

Up from 177 in Wave 1. No Docker, no node, no proof server — and green on GitHub Actions on every push.

Every suite, by name

22 files · 388 tests



Playwright, the hosted pages end to end	15
Devnet integration, with real proofs	4
CI on every push	green

The simulator runs the compiled circuits without proving — the same code path the browser sandbox uses, which is why the sandbox can refuse a fraud for real.

The adversarial suite

Sixty-five scenarios — at least one for every one of the 39 refusal codes the contract can raise — plus the lying witness: when no real Merkle path exists the witness hands back a dummy one, so the refusal comes from the circuit's assert and not from a JavaScript exception. That distinction is the test.

The property model

fast-check drives random lifecycles against a reference model. The invariants: at most one active pledge per invoice, a settled invoice is never pledgeable again, and proceeds are paid exactly once to the correct payee.

Negative controls

Threshold decryption is tested with every pair of keyholders — and with one share alone, and with the wrong auditor key, both of which must fail. An opened record that does not recompute the ledger's marker is a failure, not a warning.

The documented leaks

A whole suite reproduces the leaks and griefing vectors the threat model admits, and another asserts value by value what each circuit publishes. "Not defended" is a measured statement rather than an assumption. Every item on slide 16 has a test here.

Show your working.

Nothing here is typed by hand. It comes from the files the command-line tool wrote while it deployed two registries and ran the scenario against the first, all of it in **PROOF.md**.

Deployed, twice

PREVIEW · PUBLIC TESTNET

A · Stock & Foil	74fe0bf9…8a0fad
Deploy tx	c61e4a58…3e286d
Block	1320927
Deployed	16 Oct 2026, 10:24 UTC
B · Harbour	2d8f8f23…0cccbd
Deploy tx	c61851ae…53797a
Deployed	16 Oct 2026, 12:03 UTC
Transactions each	11

A public Midnight testnet anyone can query without asking us for anything. Eleven transactions apiece: a constructor carrying six verifier keys, then one maintenance transaction per remaining circuit, because sixteen keys do not fit in one.

And a local devnet: a node, an indexer and a proof server on one laptop, reproducible by anyone with Docker and no faucet. It shows the code runs but proves nothing to a stranger, and **PROOF.md** labels it that way.

The run

PREVIEW

Steps	33
Transactions	24
Blocks	1321121 → 1321369
Refusals recorded	5
Off-chain checks	4
Disclosure	2 of 3, verified
Whole run cost	24 SPECK

Two `NOT_ACKNOWLEDGED`, then `ALREADY_ENCUMBERED`, `NOT_PAYEE` and `BELOW_FLOOR`. None has a transaction id on any explorer, because none exists: the contract asserts while the transaction is being built.

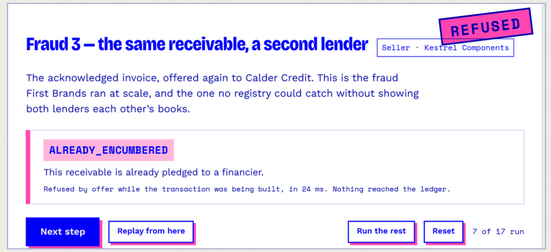
What a reviewer runs

```
npm test
388 tests, no Docker, no node,
no proof server. Same on CI.

npm run verify:onchain
field by field against the evidence:
119 checks pass, 4 later entries
noted by name. No wallet.
```

Zero-install sandbox

The web app runs the real compiled circuits in the browser tab. No wallet, no install, no chain, about thirty seconds. Every refusal on screen is the contract's own assert, not a message the page invented.



The double pledge, refused in the browser tab in 24 ms, with nothing reaching the ledger.

Re-check it without us

verify needs no wallet, no mnemonic, no proof server and no proving keys. It reads the deployed contract through the public indexer, re-derives the projection the SDK uses, and re-checks the evidence's own claims against the chain.

Who pays for a refusal.

The code is Apache-2.0 and always will be, so self-hosting is permitted and expected. The revenue sits in the *operator* role — running the instance, holding one of the three disclosure keys, answering for uptime — never in a licence.

The pot the fee comes out of

every figure published by somebody else

€4.04 tn	global factoring volume, 2025, up 3.7% — Europe €2,658bn	FCI world industry statistics, 5 May 2026
£21.2 bn	UK invoice-finance and ABL advances outstanding, 34,000 businesses	UK Finance, 13 Mar 2025
$537 bn	US asset-based-lending commitments, 2024; US factoring ~$148bn	SFNet 2025 Market Sizing Study
~$5 bn	a year, the `modelled` cost of duplicate financing: 1% of a $5tn market taken as susceptible, 10% of those as losses. No measured number exists, and we say so first	ICC UK, via GTR, Oct 2022
1 in 4	attempts even confident European receivables lenders think they miss; 89% say fraud rose	Lenvi Riskfactor survey, via FCI

The headroom, the unit, and who signs first

A factor charges **1–5% of invoice value** per roughly thirty days: on a €20,000 invoice, **€200–€1,000** of fee income, so a **$2** registration is 0.2–1% of that. A government registry already charges for this act — NZ's PPSR takes **NZ$14** to register and **NZ$2** to search.

Small factor	~£20m of advances · 25,000 pledges a year	**$50k**
Mid-market ABL	~£200m · 150,000 a year — the realistic first paying tier	**$300k**
A platform	PrimeRevenue alone: $300bn+ a year across 100+ funders, ~15m invoices	**$4m+**

The first signature comes from a mid-market specialty receivables lender — £20m–£500m of advances, one credit committee, a written-off fraud in living memory — and from its **Head of Credit**.

Three models, in the order we would sell them

B	operator subscription — first, because it is the one a Head of Credit can sign without a procurement cycle	$30k–120k / yr
A	per registration, paid by the financier taking the pledge; searching and refusal free	$1–3 / pledge
C	consortium membership, with governance seats and keyholders drawn from the members	$50k–250k / member

C is the sale the Wave 1 review correctly called hard, and nothing above it depends on reaching it. Lenders would share one instance for governance — admission, who holds the keys, who answers in an insolvency — not to detect duplicates.

what is true today, stated plainly

No customer, no pilot, no reply yet.

A written contact pack is going out to **FCI**, **SFNet** — whose own February 2026 Fraud Task Force report names First Brands' double-pledging — **ITFA**, **ICC United Kingdom** and the **EUF**, and to **Finverity**, **Evolution Credit Partners**, **White Oak Commercial Finance** and **Tokio Marine HCC**. Every address was read off the organisation's own site; none is a guessed pattern. **Nobody has answered yet, and we will not claim it until somebody has.**

The hard gate is an external audit. No regulated lender pays for an unaudited registry that holds its collateral position, and no amount of outreach substitutes for it.

Honest limits.

Every item below has a test that reproduces it, so "not defended" is a measured statement rather than an assumption. Three of them were found this wave by running the thing on a real network.

- **It has never seen a real invoice.** Every run is synthetic: our personas, our numbers. No lender has used this, no pilot has run, and nothing here has been tested against a real book.
- **One registry cannot see another.** A ledger cannot read another ledger, so a second registry will accept a receivable already pledged in the first — Harbour did, on preview, at block 1322275. Only the lender's own cross-registry check stops the advance, and *registerPeer* is a declaration, not a proof that a peer is honest.
- **Collusion still works.** A buyer who colludes with a supplier can acknowledge an invoice that does not exist. The registry makes that acknowledgment non-repudiable; it does not prevent it.
- **Settlement is public.** The settlement token is unshielded, so amounts and payer and payee addresses are visible at payment time. Contract custody of shielded coins is blocked upstream.
- **The operator is trusted twice.** It decides who is admitted, and as the deployer it holds the contract's maintenance authority — a strictly larger power. Production should move that authority to a multi-party key, or burn it.
- **The keys are dealt, not generated.** A ceremony splits the disclosure key rather than a distributed protocol, and **two colluding keyholders** reconstruct it off-chain with no public request at all.
- **Single-record approvals can still be pooled.** A keyholder's share is bound to the record, so two requests naming one record with one approval each open it. *openCase* closes this for an insolvency by binding shares to a case reference; the single-record path is unchanged.
- **A certificate names a lender it cannot bind.** The holder tags on its locked slots come from the seller, and no circuit can tie them to that lender. The lender recomputes every tag from public state before it advances.
- **A borrowing base shows its markers, and needs all four slots.** Publishing the markers it locks is what makes the lock verifiable. A partly filled pool passes the simulator and fails on a live network: the proof server rejects the zeroed curve point an unused slot leaves behind.
- **A seller can leak its own records.** Reusing a sealing scalar makes two records comparable to anyone, and re-offering with the same scalar overwrites the earlier one. The SDK draws a fresh scalar every time; the discipline is still the caller's.
- **A settled, unfinanced book is linkable.** An opened record reveals the supplier's id, and the seller payee tag derives from it, so every settled-but-unfinanced receivable of that supplier becomes recognisable without further approval.
- **Timing correlates.** An observer can link an acknowledgment to a later offer by when each happened.
- **Sealing rests on Poseidon.** The platform does not promise the hash stays stable across upgrades, which is why every record carries a version.
- **Preview, not mainnet.** Two registries run on Midnight preview, a public testnet anyone can query. We attempted mainnet three times through a sponsoring wallet: each transaction was proved, balanced and accepted by the node, and each was invalidated in its pool 75 seconds later. Nothing landed, nothing was spent, and the cause we could not exclude is on the sponsor's side of the line — slide 17.
- **Not audited.** An internal security review, with findings accepted on the record — one of them, F-08, is what case-scoped disclosure answers. No external audit.

The threat model names the attacker behind each of these, the property it breaks and the test that reproduces it. Both documents ship with the repository.

docs/THREAT-MODEL.md · docs/PRIVACY-BOUNDARY.md
contract/test/limits.test.ts · contract/test/transcript.test.ts

What you told us, and what we did.

Three notes came back from Wave 1. Two are answered here; the third we tried, and this is how far it got.

Every change since the Wave 1 submission, item by item, with what it is and why, is at **/whats-new** on the site — ten of them, this one included.

Communication — answered

> *"The presentation would be even better if it made it clearer who stands to benefit from using this."*

Four named seats, first thing

The credit officer at a factoring company, the supplier's CFO, the buyer's accounts-payable team and the insolvency practitioner — each with what they are stuck with today, what the registry hands them and the document they stop sending.

slide 03 · four workspaces in /app · the site's /#who

Product — answered

> *"The catch is scale: it only works once many competing financiers join the same registry, a consortium sale that's hard for a startup."*

Two registries, and a check across them

Two deployments on preview, two operators, two ceremonies, each declaring the other a peer on chain. A lender asked both about one receivable *before* the second had seen it: its own book said FREE, the peer said ENCUMBERED, and it declined on that bit. It runs live in the reviewer's browser.

slide 07 · the site's /#check · the gap it leaves, slide 16

Business Development — attempted, not landed

> *"Since you'll soon be able to freely deploy contracts to the mainnet, please be sure to deploy your product there."*

Not on mainnet. Here is exactly why

Three attempts, through a wallet that pays the DUST fee from its own pool, ours being on Cardano with no route there. Each was proved, balanced and taken by the node, then marked *Invalid* 75 seconds later. Nothing landed, nothing was spent. The node prices all eleven at **0.002008 DUST** against the **1.000000** held, so fees are out, and so are block limits and every static cause. The sponsor's DUST input moved under us.

docs/MAINNET.md §3b · the measurements and the exclusions

Also new this wave

none of it was in the Wave 1 submission

- **Four circuits, 12 → 16.** registerPeer, acceptBatch, openCase, approveCase.
- **Tests 177 → 388,** and CI on every push.
- **Screens rebuilt to read in one still frame.**
- **Fee sponsorship:** a wallet with 0 DUST transacted.
- **A second ceremony and deployment,** 11 txs each.
- **A business model, adoption path and outreach list.**

Next, in order: mainnet, on a steadier sponsor or DUST of our own · an external audit, the gate before any lender pays · one pilot · then an operator who is not us.

Where to look

The app, no wallet, 30 seconds	`stock-and-foil.vercel.app`
Every change this wave	`/whats-new`
The code, Apache-2.0	`github.com/RaYYeR220/stock-and-foil`
Every transaction, with links	`PROOF.md`

Every invoice can be pledged once.
Prove it without revealing it.